University of Ferhat Abbas
Faculty of Technology
Department of Automation and Intelligent Systems

Bachelor Thesis

Supervised and Unsupervised Machine Learning
A Practical Study With Python Implementation

Presented by: Bouchelouche Mansour
Supervisor: M. Bektache A

Academic Year:2024-2025

Table of Contents

CHAPTER I: INTRODUCTION TO AI....5
1. Introduction:....5
2. What is AI?....5
3. What is Machine Learning (ML):....5
CHAPTER II: SUPERVISED LEARNING....7
2. What Is Supervised Learning?....7
2.1 Classification:....7
2.2 Regression:....8
2.3 Supervised Learning Algorithms:....9
2.3.1 Support Vector Machines (SVM):....9
2.3.1.1 Understanding SVMs:....9
2.3.1.2 how SVMs work:....9
2.3.1.3 Types of SVMs:....10
2.3.1.4 Applications of SVMs:....11
2.3.1.5 Advantages and disadvantages of SVMs:....11
2.3.1.6 SVMs In Practice:....11
Example 1 (Classification):....11
Example 2 (Classification):....13
Example 3 (Regression):....14
2.3.2 Naive Bayes:....15
2.3.2.1 Understanding Naive Bayes:....15
2.3.2.2 How Naive Bayes work:....16
2.3.2.3 Types of Naive Bayes classifiers:....16
2.3.2.4 Applications of the Naive Bayes classifier:....16
2.3.2.5 Advantages and disadvantages of Naive Bayes classifier:....17
2.3.2.6 Naive Bayes in practice:....17
2.3.3 Linear Discriminant Analysis (LDA):....18
2.3.3.1 Understanding LDA:....18
2.3.3.2 How LDA works:....18
2.3.3.3 Applications of LDA:....18
2.3.3.4 Advantages and disadvantages of LDA:....19
2.3.3.5 LDA in practice:....19
2.3.4 Decision Trees (DT):....20
2.3.4.1 Understanding Decision Trees:....20
2.3.4.2 How Decision Trees work:....20
2.3.4.3 Types of Decision Trees:....21
2.3.4.4 Applications of Decision Trees:....21
2.3.4.5 Advantages and disadvantages of Decision Trees:....22
2.3.4.6 Decision Trees in practice:....22
Example 1 (Classification):....22
Example 2 (Regression):....22
2.3.5 Random Forests:....23
2.3.5.1 Understanding Random Forests:....23
2.3.5.2 How Random Forest Works:....23
2.3.5.3 Applications of Random Forests:....24

2.3.5.4 Advantages and Disadvantages of Random Forests:....24
2.3.5.5 Random Forests in practice:....25
Example 1 (Classification):....25
Example 2 (Regression):....25
2.3.6 K-nearest Neighbors (K-NN):....26
2.3.6.1 Understanding K-NN:....26
2.3.6.2 How K-NN works:....26
2.3.6.3 Applications of the K-NN algorithm:....27
2.3.6.4 Advantages and Disadvantages of K-NN algorithm:....27
2.3.6.5 K-NN in practice:....27
Example 1 (Classification):....27
Example 2 (Regression):....28
2.3.7 Neural Networks (NN):....29
2.3.7.1 Understanding Neural Networks:....29
2.3.7.2 Types of Neural Networks:....30
2.3.7.3 Applications of Neural Networks:....30
2.3.7.4 Advantages and disadvantages of Neural Networks:....30
2.3.7.5 Neural Networks in practice:....31
Example 1 (Classification):....31
Example 2 (Regression):....31
CHAPTER III: Unsupervised Learning....33
3. What is Unsupervised Learning?....33
3.1 Clustering:....33
3.1.1 Definition of Clustering:....33
3.1.2 K-Means Clustering Algorithm:....34
3.1.2.1 Understanding K-Means Clustering:....34
3.1.2.2 How K-Means work:....34
3.1.2.3 Applications of K-Means:....35
3.1.2.4 Advantages and disadvantages of K-Means:....35
3.1.2.5 K-Means in practice:....35
3.2 Association:....36
3.2.1 Definition of Association:....36
3.2.2 Apriori Algorithms:....37
3.2.2.1 Understanding Apriori algorithm:....37
3.2.2.2 How Apriori algorithm works:....37
3.2.2.3 Applications of Apriori algorithm:....38
3.2.2.4 Advantages and disadvantages of Apriori algorithm:....38
3.2.2.5 Apriori algorithm in practice:....38
3.3 Dimensionality Reduction:....40
3.3.1 Definition of Dimentionality Reduction:....40
3.3.2 Principle Component Analysis (PCA):....41
3.3.2.1 Understanding PCA:....41
3.3.2.2 How PCA works:....41
3.3.2.3 Applications of PCA:....41
3.3.2.3 Advantages and disadvantages of PCA:....42
3.3.2.4 PCA in practice:....42
Conclusion:....44

References.....45

CHAPTER I: INTRODUCTION TO AI

1. Introduction:

In recent years, technology has advanced at an exceptional pace, transforming the way we live, work, and interact with the world. These changes have opened the door to new possibilities in various fields, especially in data analysis and intelligent systems. As the amount of data continues to grow, the need for efficient methods to interpret and use this information has become more important than ever.

2. What is AI?

AI is an area of computer science that aims to make machines do intelligent things, learn, and solve problems—similar to the natural intelligence of humans and animals. Alan defines it as follows: “If there is a machine behind a curtain and a human is interacting with it (by whatever means, e.g., audio or via typing, etc, and if the human feels like he/she is interacting with another human, then the machine is artificially intelligent” [1].

In AI, an intelligent agent receives information from the environment, performs computations to decide what action to take in order to achieve goals autonomously. In traditional systems, its behavior is improved through explicit programming, where developers manually define rules. In contrast, learning-based systems adapt automatically by analyzing data [2].

AI doesn’t mean building an extraordinarily intelligent machine that can solve any problem in no time, it means building a machine that is capable of human-like behavior. However, just building machines that mimic humans does not sound very interesting. Whenever we speak of AI, we mean machines that are capable of performing one or more of these tasks: understanding human language, performing mechanical tasks involving complex maneuvering, solving computer-based complex problems possibly involving large data in a very short time, and reverting back with answers in a human-like manner, etc [1].

3. What is Machine Learning (ML):

Machine learning is basically a set of mathematical algorithms, it is an important subset of AI (Figure 1.1), and it is the science that aims to teach computers, or machines, to learn from data and to analyze data automatically, without human intervention (Figure 1.2). It includes a set of mathematical algorithms that can make a decision or, more accurately, predict the result for a given set of data [1].

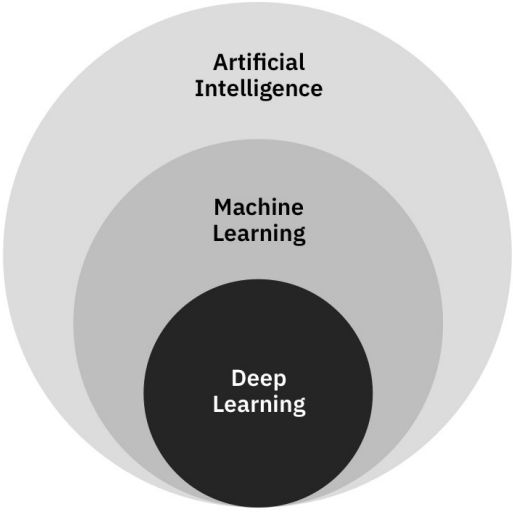

Figure 1.1: Machine Learning is a subset of Artificial Intelligence [3].

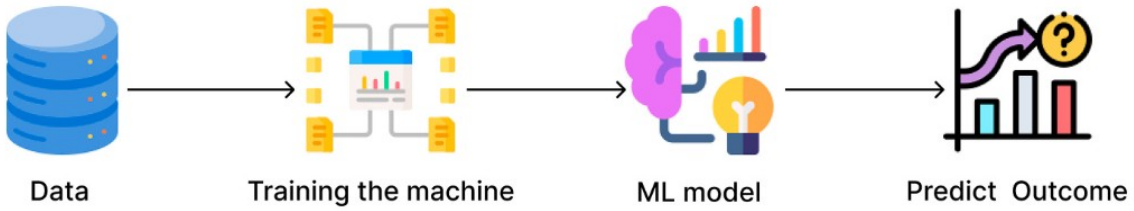

Figure 1.2: The meaning of machine learning [4].

Machine learning can be divided four different categories: supervised learning, unsupervised learning, Semi-supervised learning and reinforcement learning [2].

This thesis will focus on the application of machine learning techniques, particularly supervised and unsupervised learning.

CHAPTER II: SUPERVISED LEARNING

2. What Is Supervised Learning?

Supervised learning is a branch of machine learning wherein the algorithm learns from input features associated with known output labels or target values, enabling it to predict or classify new, unseen data. Supervised learning relies on a dataset containing input-output pairs to train the algorithm. This concept can be likened to learning under the guidance of a supervisor. Generally, supervised learning proves more effective when labeled datasets are available than other learning methods. Its applications span various real-life scenarios, including fraud detection (e.g., distinguishing between fraudulent and legitimate transactions), sales forecasting (e.g., predicting high, medium, or low sales), and email categorization (e.g., identifying spam emails). Table 1.1 depicts an example of a labeled dataset containing the diabetes diagnostic measurements, where the last feature [4, p. 3].

Supervised learning has two main branches: C*lassification* and *Regression*.

2.1 Classification:

Classification means assigning input data samples into predefined categories or classes. Drawing from previous learning experiences, a classification algorithm typically identifies data samples within the input dataset and assigns them to specific classes (Figure 2.3) [5].

The best-known classification example in the field of machine learning is Iris flower classification, which is to classify iris flowers among three species (Setosa, Versicolor, or Virginica) based on the measured length and width of the sepals and petals [2].

The most popular classification algorithms are the following: support vector machines, naive Bayes, linear discriminant analysis, decision trees, k-nearest neighbors algorithm and neural-networks [2].

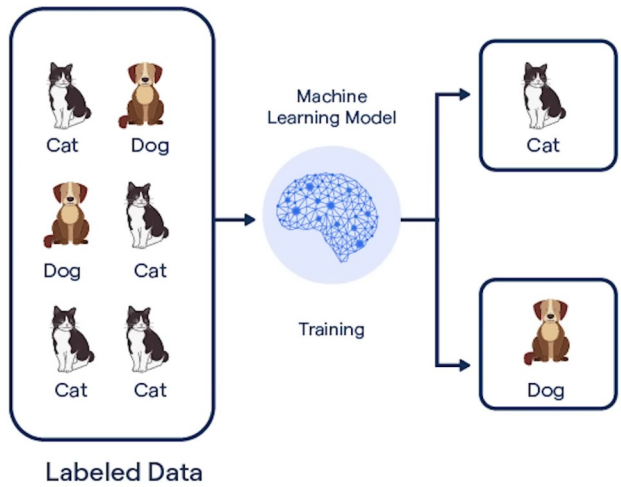

Figure 2.3: Supervised Learning (classification) [6].

2.2 Regression:

regression means fitting the data with a mathematical model using a technique called *least squares fitting.* Regression can be divided into linear regression and nonlinear regression [2]:

- **Lear regression:**
  Fits the data with a straight line ($f(x)=a.x+b$), where (a) is the slope, and (b) is the intercept, Figure 1.4. For a given dataset, we calculate the sum of the squares of the errors (e_i), calculate the distances between the data points and the straight line, and adjust the slope (a) and the intercept (b) of the straight line until we have reached the smallest sum of the squares (x^2) [2].

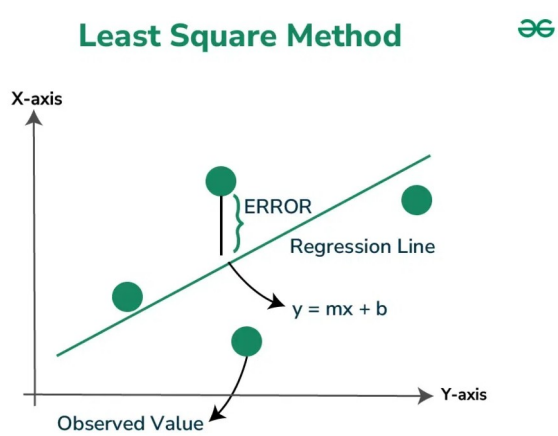

Figure 1.4: Least Square Method [7].

- **Non-linear regression:**
  Fits the data with more complicated mathematical models, such as exponentials and polynomials [2]; see Figure 1.5.

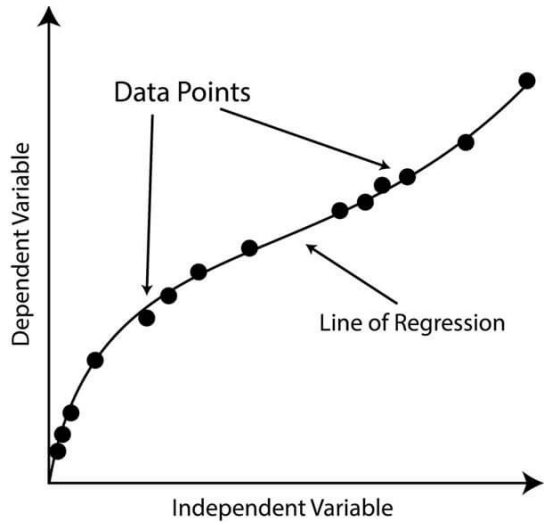

Figure 1.5: Non-Linear Regression [8].

2.3 Supervised Learning Algorithms:

2.3.1 Support Vector Machines (SVM):

2.3.1.1 Understanding SVMs:

Support vector machines are a set of supervised learning methods used for classification and regression [9], it was developed in 1990s, it's one of the most robust prediction methods, based on statistical learning framework [1].

2.3.1.2 how SVMs work:

SVMs distinguishes between two classes by finding the optimal hyperplane that maximizes the margin between the closest data points of opposite classes. The number of features in the input data determine if the hyperplane is a line in a 2-D space or a plane in a *n-dimensional* space. Since multiple hyper-planes can be found to differentiate classes, maximizing the margin between points enables the algorithm to find the best decision boundary between classes. This, in turn, enables it to generalize well to new data and make accurate classification predictions. The lines that are adjacent to the optimal hyperplane are known as support vectors as these vectors run through the data points that determine the maximal margin [10].

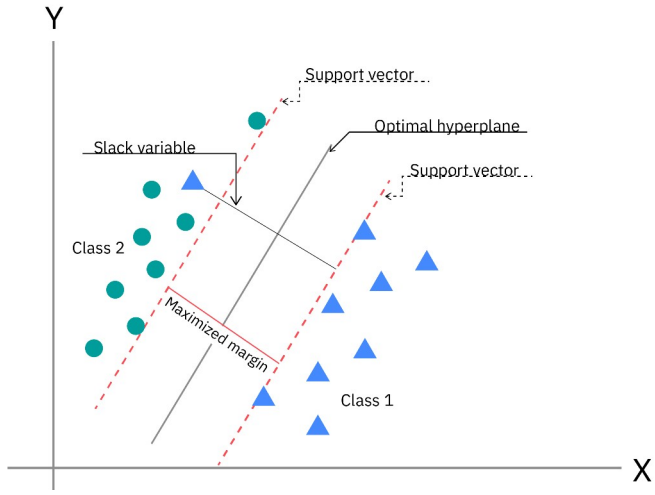

Figure 1.6: Support Vector Machines [10].

The SVM algorithm is widely used in machine learning as it can handle both linear and nonlinear classification tasks. However, when the data is not linearly separable, kernel functions are used to transform the data higher-dimensional space to enable linear separation. This application of kernel functions can be known as the "kernel trick", and the choice of kernel function, such as linear kernels, polynomial kernels, radial basis function (RBF) kernels, or sigmoid kernels, depends on data characteristics and the specific use case [10].

2.3.1.3 Types of SVMs:

- **Linear SVMs:**

  Linear SVMs are used with linearly separable data; this means that the data do not need to undergo any transformations to separate the data into different classes. The decision boundary and support vectors form the appearance of a street [10].

- **Non-linear SVMs:**

  Most real-world data isn't linearly separable. To handle this, nonlinear SVMs use pre-processing to map data into a higher-dimensional space where it becomes separable. However, this adds complexity and can lead to overfitting or heavy computation. The **kernel trick** addresses this by using kernel functions to perform the transformation efficiently, without directly computing the new space [10].

  Common kernels include:

  - Polynomial
  - Radial Basis Function (RBF or Gaussian)
  - Sigmoid [10].

- **Support vector regression (SVR):**

  Support Vector Regression (SVR) extends SVM to regression tasks with continuous output. It finds a maximum-margin hyperplane, similar to linear SVMs. Unlike linear regression, SVR automatically learns relationships between variables, making it well-suited for time series prediction [10].

2.3.1.4 Applications of SVMs:

- **Text classification**: It's commonly used in natural language processing for sentiment analysis and spam detection [10].
- **Image classification:** SVMs are applied in image classification tasks such as object detection and image retrieval, and it's useful in security domains [10].
- **Bio-informatics:** it is used for protein classification, gene expression analysis and Disease diagnosis [10].

2.3.1.5 Advantages and disadvantages of SVMs:

Advantages:

- Effective in high dimensional spaces.
- Still effective in cases where number of dimensions is greater than the number of samples.
- Memory efficient.
- Versatile: different kernal functions can be specified for the decision function [9].

Disadvantages:

- Choosing the right kernal and regularization is crucial when the number of features is much greater than the number of samples.
- SVMs do not directly provide probability estimates, these are calculated using an expensive five-fold cross-validation [9].

2.3.1.6 SVMs In Practice:

Example 1 (Classification):

To illustrate the basic idea of how Support Vector Machines (SVM) work, we'll begin with a simple binary classification problem: predicting a person's gender based on their *physical attributes*. In this example, we'll use supervised learning, where the model learns from a set of labeled training data.

- **Importing the SVM module:**

  The process begins with importing the SVM classifier from the `scikit-learn` library, which is widely used for machine learning in Python.

```
from sklearn import svm
```

- **Preparing the training data:**

  Defining our **feature matrix (X)**, which contains four training samples. Each sample is a list of three numerical features: *height* (cm)*, weight* (kg), and *shoe size* (UK size).

```
X = [[170,70,10],[180,80,12],[170,65,8],[160,55,7]]
```

  These are the *independent variables* (or features) that the model will use to learn patterns.

  Then defining the **target vector (y)**, which contains the corresponding **labels** (also called classes or outputs) for each training sample. In this case, we encode gender as *0* for male and *1* for female.

```
y =[0, 0, 1, 1]
```

  Here, each label in y is associated with the same-indexed feature vector in X.

- **Initializing the SVM classifier:**

  Creating an instance of the SVC class, which stands for *Support Vector Classifier.* This initializes an SVM model with the default hyper-parameters.

```
clf = svm.SVC()
```

- **Training the model:**

  Fitting the classifier to our training data using the ***`fit()`*** method. This is where the SVM learns the optimal decision boundary that best separates the two classes in feature space.

```
clf.fit(X, y)
```

- **Making a prediction**

  Once trained, the model can be used to make predictions on new, unseen data. We provide a new data point with the same structure (height, weight, shoe size), and the model returns the predicted class.

```
p = clf.predict([[160, 60, 7]])
print(p)
```

- **Output:**

```
[1]
```

  The output is ***`[1]`***, the model has classified the individual as female.

Example 2 (Classification):

In this example, we aim to classify the type of Iris flower based on its features using a Support Vector Machine (SVM). The *Iris dataset* is a classic benchmark in machine learning, containing three classes of flowers: *Setosa, Versicolor*, and *Virginica*. Each instance is described by four numerical features: *sepal length, sepal width, petal length,* and *petal width.*

To simplify the visualization, we will use only the *sepal length* and *sepal width* as input features.

- **Importing required libraries:**
  - `numpy` for handling numerical operations
  - `matplotlib.pyplot` for plotting (used later for visualization)
  - `sklearn.svm` for the SVM classifier
  - `sklearn.datasets` to load the Iris dataset

```
import numpy as np

import matplotlib.pyplot as plt
from sklearn import svm, datasets
```

- **Loading data:**
  We load the Iris dataset using ***`datasets.load_iris()`***.

```
iris = datasets.load_iris()
```

  From the dataset, we extract only the first two *features* (columns) for simplicity: *sepal length* and *sepal width.*
  y vector contains the corresponding *target labels*, encoded as 0 (*Setosa*), 1 (*Versicolor*), and 2 (*Virginica*).

```
X = iris.data[:,:2]
y = iris.target
```

- **Creating and training the model:**
  Creating an SVM classifier with a *linear kernel*, suitable for linearly separable classes.
  The model is trained using the features X and the target labels y.

```
clf = svm.SVC(kernel='linear')
clf.fit(X, y)
```

- **Making prediction:**
  After training, we use the model to predict the class of a new flower with *sepal length 5.4 (cm)* and *sepal width 3.2 (cm).*

```
prediction = clf.predict([[5.4, 3.2]])
print(prediction)
```

- **Output:**

```
[0]
```

The SVM model predicts that this unseen data sample is a *Setosa* flower (label 0).

- **Visualization:**

To better understand how the model separates the classes, we generated a 2D plot of the decision boundaries using the two selected features.

Note: The code for visualization is omitted for clarity, as it does not contribute to the explanation of the training and prediction process

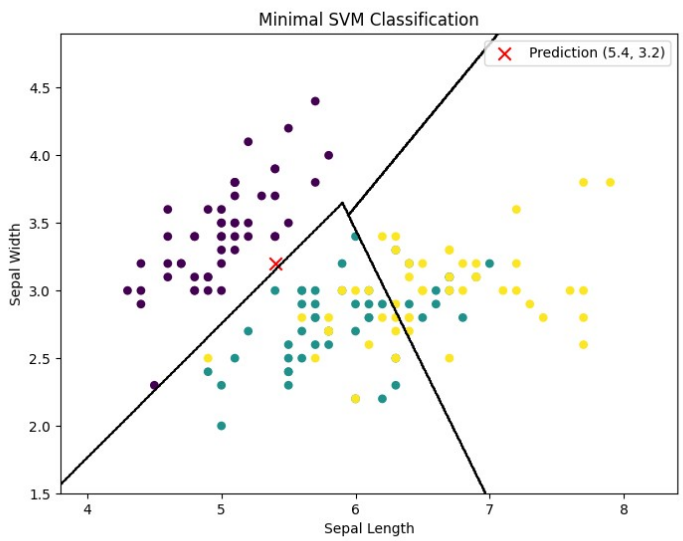

Figure 1.7: SVM Classification.

Example 3 (Regression):

- Importing required libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVR
```

- Preparing the data:

```
X = np.arange(0, 10, 0.5).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])
```

- Training the model:

  Here the ***RBF*** kernel is used because it is optimal for dealing with noisy sine wave data.

```
clf = SVR(kernel='rbf', C=100, epsilon=0.1)
clf.fit(X, y)
```

- Making prediction:

```
X_test = np.linspace(0, 10, 100).reshape(-1, 1)
y_pred = clf.predict(X_test)
```

- Visualization:

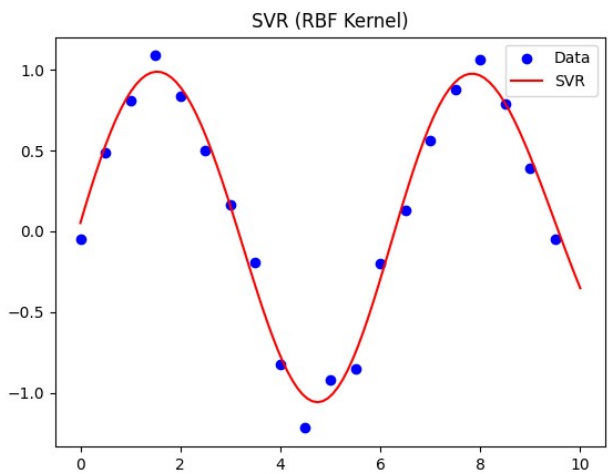

Figure 1.8: Support Vector Regression (SVR) model with an RBF kernel.

The curve represents the model's predictions, where the error is minimized across the dataset, demonstrating its ability to predict continuous values.

2.3.2 Naive Bayes:

<u>2.3.2.1 Understanding Naive Bayes:</u>

Naive Bayes is another popular supervised learning algorithm which applies Bayes' theorem with the naive assumption of the probabilities of features for a given dataset. Naive assumption means every feature is independent of the others [2],it does not learn which features are most important to differentiate between classes [11].

Bayes' theorem, also known as Bayes' Rule, allows us to "invert" conditional probabilities. As a reminder, conditional probabilities represent the probability of an event given some other event has occurred, which is represented with the following formula [2]:

$$P(C/X)=\frac{P(C)*P(X/C)}{P(X)}$$

Where:

Formula	Meaning
P(C/X)	the posterior, the probability of class (C) given features (X)
P(C)	the prior, the probability of class (C)
P(X/C)	the likelihood, the probability of features (X) for a given class (C)
P(X)	the prior, the probability of features (X)

Table 2.1: Bayes Theorem Meaning [2].

2.3.2.2 How Naive Bayes work:

To train a naive Bayes classifier, you just need to calculate all the probabilities $P(C)$ of all the classes (types of flowers in this case) for the given features (septal and petal' length and width) and evidence $P(X)$. For a given dataset, $P(C)$ and $P(X)$ are constants, So, they are saved for future use. Given a sample X, the likelihood $P(X/C)$ will be calculated, and the posterior $P(C/X)$ can be calculated [2].

2.3.2.3 Types of Naive Bayes classifiers:

Gaussian Naive Bayes:

- Typically used with Gaussian distributions (normal distributions) and continuous variables.
- This model is fitted by finding the mean and standard deviation of each class [11].

Multi-nomial Naive Bayes:

- Assumes that the features are from multi-nomial (interger-count) distributions.
- Useful when using discrete data such as frequency counts [11].

Bernoulli Naive Bayes:

- Suitable for binary feature data [11].

2.3.2.4 Applications of the Naive Bayes classifier:

- **Spam Email Filtering:** Classifies emails as spam or non-spam based on features.

- **Text Classification:** Used in sentiment analysis, document categorization, and topic classification.
- **Sentiment analysis:** A type of text classification used in marketing to gauge opinions on products and brands.
- **Mental state prediction:** Helps identify cognitive states, aiding research on brain injuries [11].

2.3.2.5 Advantages and disadvantages of Naive Bayes classifier:

Advantages:

- **Simple & Fast** – Easy to implement and works well with limited data.
- **Low Computational Cost** – Scales well with high-dimensional data.
- **Works with Small Data** – Performs well even with scarce data.
- **Handles Categorical & Numerical Data** – Supports various data types (Gaussian, Multinomial, Bernoulli) [11].

Disadvantages:

- **Independence Assumption** – Assumes features are independent, which is unrealistic in some cases.
- **Zero Probability Problem** – Zero probability for unseen feature values (fixed with Laplace smoothing).
- **Sensitive to Irrelevant Features** – Noisy features can harm performance.
- **Not Ideal for Complex Relationships** – Struggles with feature interactions (e.g., XOR patterns) [11].

2.3.2.6 Naive Bayes in practice:

We are still going to use the Iris dataset in this method as well. It remains a great choice for demonstrating basic classification algorithms due to its simplicity and structured features.

- Importing the necessary tools:

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
```

- Loading the dataset:

```
X, y = load_iris(return_X_y=True)
```

- Creating and training the model:

```
clf = GaussianNB()
clf.fit(X, y)
```

- Making a prediction:

```
p = clf.predict([[5.0, 3.4, 1.5, 0.4]])
print(p)
```

- Output:

```
[0]
```

The Naive Bayes algorithm predicts that this data sample belongs to the *Setosa* class (label 0).

2.3.3 Linear Discriminant Analysis (LDA):

2.3.3.1 Understanding LDA:

The goal of LDA is to project a dataset onto a lower-dimensional space to separate them better into different classes and to reduce computational costs [2].

Linear Discriminant Analysis works well even when the number of samples in each class is not equal. It has been tested on random data and aims to separate classes by maximizing the difference between them while minimizing the variance within each class [12].

LDA is thus often applied to enhance the operation of other learning classification algorithms such as Decision trees, Random Forest, or SVMs [13].

2.3.3.2 How LDA works:

LDA works by identifying a linear combination of features that separates or characterizes two or more classes of objects or events. LDA does this by projecting data with two or more dimensions into one dimension so that it can be more easily classified. The technique is, therefore, sometimes referred to as dimensionality reduction. This versatility ensures that LDA can be used for multi-class data classification problems, it basically projects the data from one dimension linearly into another dimension [13].

Apart from LDA, there is another nonlinear discriminant analysis methods such as:

- Quadratic discriminant analysis (QDA).
- Flexible discriminant analysis (FDA).
- Regularized discriminant analysis (RDA) [13].

2.3.3.3 Applications of LDA:

- Credit risk assessment.
- Disease diagnosis.
- Customer segmentation.
- Campaign optimization.

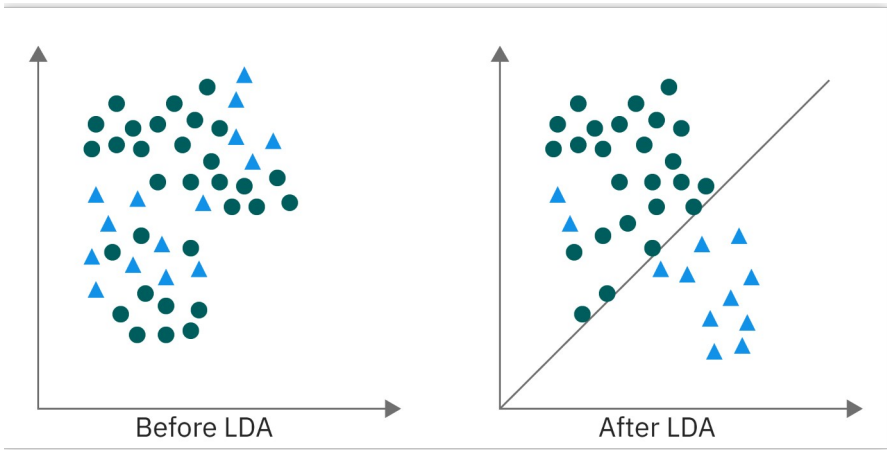

Figure 1.9: Data before and after using LDA [13].

2.3.3.4 Advantages and disadvantages of LDA:

Advantages:

- **Simplicity and Efficiency**: LDA is simple to implement and computationally efficient.
- **Handling High-Dimensional Data**: It works well when features outnumber training samples.
- **Dealing with Multi-collinearity**: can work with correlated features by projecting them into a lower-dimensional space while preserving important information [13].

Disadvantages:

- **Shared Mean Distributions**: It faces challenges when classes share similar means, making separation difficult.
- **Not Suitable for Unlabeled Data**: It is a supervised learning method, requiring labeled data, unlike PCA, which can handle unlabeled data [13].

2.3.3.5 LDA in practice:

- Importing required tools:

```
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

- Loading the dataset and training the model:

```
iris = load_iris()
X, y = iris.data, iris.target

clf = LinearDiscriminantAnalysis()
clf.fit(X, y)
```

- Making prediciton:

```
print(clf.predict([[5.0, 3.4, 1.5, 0.4]]))
```

- Output:

```
[0]
```

The LDA model predicts that this unseen data sample is a **Setosa** flower (label 0).

- Visualization:

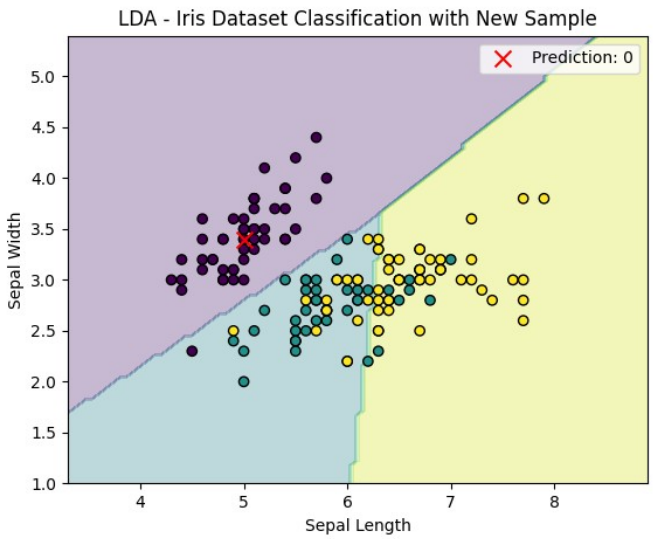

Figure 1.10: Making prediction with LDA.

2.3.4 Decision Trees (DT):

2.3.4.1 Understanding Decision Trees:

A decision tree is one of the most widely used, non-parametric, supervised learning methods, and it can be used for both classification and regression problems [2]. It has a hierarchical, tree structure, which consists of a root node, branches, internal nodes and leaf nodes [19]

2.3.4.2 How Decision Trees work:

A decision tree begins with a main question called the *root node*, which is based on a key feature of the dataset and marks the starting point of the decision-making process.
From there, the tree proceeds through a sequence of *yes/no* questions, each aimed at splitting the data into smaller subsets according to specific attributes [2].

A decision tree process start from the tree root and goes down to branches and leaves. Each branch represents an *if-then rule*. For example, the first branch of the decision tree in the Figure 1.14 represents the rule:

- If the weather is **Sunny**:
    - If the humidity is **High**, then **Don't play**.
    - If the humidity is **Normal**, then **Play**.

And so on

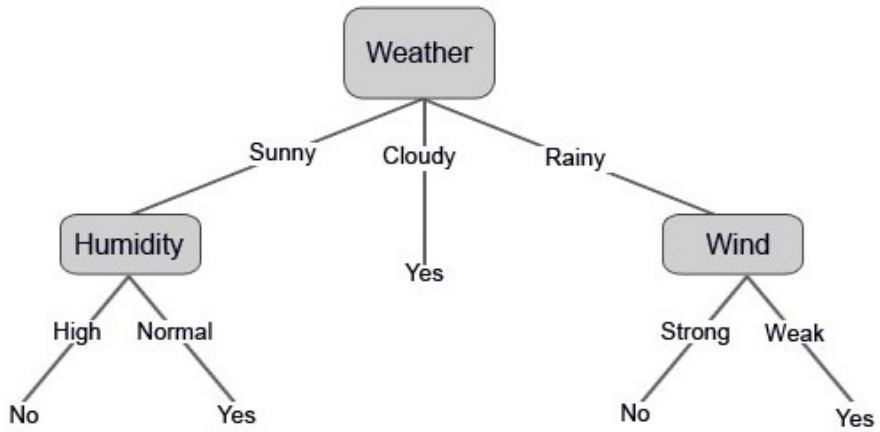

Figure 1.14: Playing Golf Decision Tree [16].

The deeper the tree is the more complex the rules and the model. The goal of a decision tree training algorithm is to create a decision tree based on a dataset and finally produce a set of rules for prediction given a sample [2].

2.3.4.3 Types of Decision Trees:

The most widely used Decision Tree algorithms include:

- **ID3** (Iterative Dichotomiser 3).
- **C4.5** (An improvement over ID3).
- **CART** (Classification and Regression Tree).
- **CHAID** (Chi-squared Automatic Interaction Detector).
- **QUEST** (Quick, Unbiased, Efficient Statistical Tree) [17].

2.3.4.4 Applications of Decision Trees:

- Load approval.
- Medical Diagnosis.
- Student performance [18].

2.3.4.5 Advantages and disadvantages of Decision Trees:

Advantages:

- **Simple and Interpretable**: Easy to understand and visualize.
- **Versatile**: Works for both classification and regression.
- **No Feature Scaling Needed**: No need to normalize data.
- **Handles Non-linear Data**: Captures complex relationships [19].

Disadvantages:

- **Over-fitting**: Can learn noise in the training data.
- **Unstable**: Small changes in data can affect the model.
- **Bias to Many-Category Features**: It focuses too much on features with many levels [19].

2.3.4.6 Decision Trees in practice:

Example 1 (Classification):

- Importing required libraries:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
```

- Loading dataset and training the Decision Tree model:

```
X, y = load_iris(return_X_y=True)

clf = DecisionTreeClassifier()
clf.fit(X, y)
```

- Making prediction:

```
print(clf.predict([[5.0, 3.4, 1.5, 0.4]]))
```

- Output:

```
[0]
```

The decision tree classifier also predicts that this data sample is a Setosa flower.

Example 2 (Regression):

- Importing required libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
```

- Loading data and training the Decision Tree Regressor:

```
X = np.arange(0, 10, 0.5).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

clf = DecisionTreeRegressor(max_depth=10)
clf.fit(X, y)
```

- Making prediction:

```
X_test = np.linspace(0, 10, 100).reshape(-1, 1)
y_pred = clf.predict(X_test)
```

- Visualization:

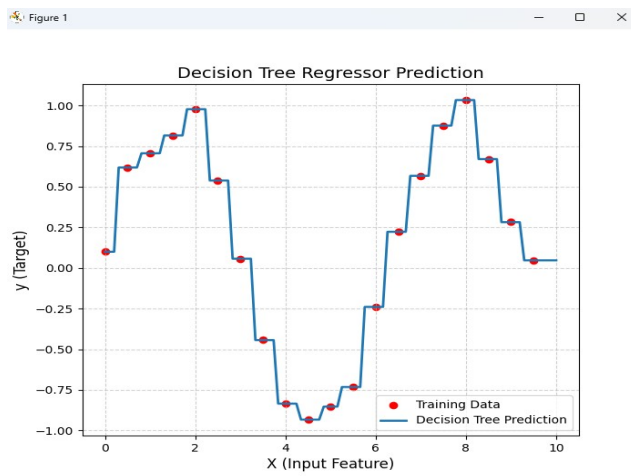

Figure 1.15: Decision Tree Regressor model.

2.3.5 Random Forests:

<u>2.3.5.1 Understanding Random Forests:</u>

A random forest is an algorithm that uses multiple decision trees, a single decision tree might not be enough for some applications, A random forest randomly creates a set of of decision trees, with each one working on a random subset of data samples [2], it can be used both for regression or classification problems [20].

<u>2.3.5.2 How Random Forest Works:</u>

Random Forest builds many decision trees using random data samples and random feature subsets. Each tree gives a prediction. For classification, the final result is based on *voting* (either by majority class or by averaging class probabilities). For regression, the final result is the *average* of the tree predictions shown in Figure 1.16. This randomness helps reduce over-fitting and improves accuracy [20].

Key Assumptions:

- Each tree works independently.
- Random samples and features are used to build trees.
- A large dataset is needed for variety.
- Combining multiple tree outputs improves prediction [22].

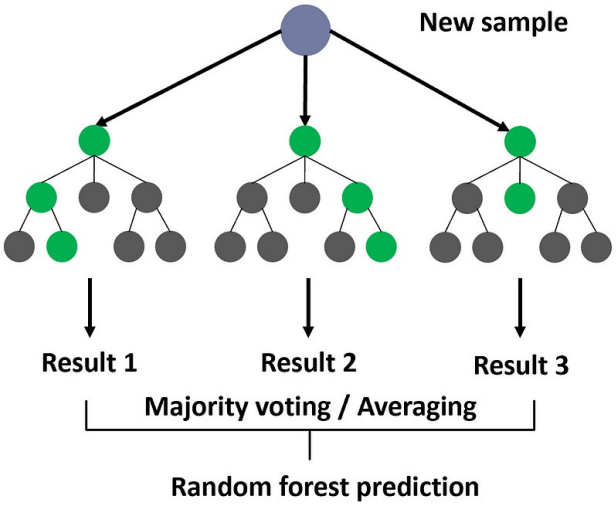

Figure 1.16: Random Forest prediction [21].

2.3.5.3 Applications of Random Forests:

- **Finance:** Used for credit risk assessment, fraud detection, and option pricing.
- **Healthcare:** Applied in computational biology for gene classification, biomarker discovery, and drug response prediction.
- **E-commerce:** Powers recommendation engines for cross-selling.

2.3.5.4 Advantages and Disadvantages of Random Forests:

Advantages:

- **High Accuracy:** Combines multiple trees for improved prediction accuracy.
- **Robust to Noise:** Less impacted by noisy data or outliers.
- **Feature Importance:** Helps identify the most relevant features.
- **Handles Data:** Can work with missing values and mixed data types [20].

Disadvantages:

- **High Cost:** Training and prediction can be computationally intensive and slow.
- **Memory Intensive:** Requires significant memory usage.
- **Less Interpretable:** Difficult to understand the decision process ("black box").
- **Potential Over-fitting:** Can still over-fit if not carefully tuned [23].

2.3.5.5 Random Forests in practice:

Example 1 (Classification):

- Importing required libraries:

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
```

- Loading the dataset and training the classifier:

```
X, y = load_iris(return_X_y=True)

clf = RandomForestClassifier()
clf.fit(X, y)
```

- Making prediction:

```
print(clf.predict([[5.0, 3.4, 1.5, 0.4]]))
```

- Output:

```
[0]
```

Random forest also predicts that the given data sample is a s*etosa* flower.

Example 2 (Regression):

In this example we will be using Random Forest Regressor.

- Importing required libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
```

- Preparing the data and training the random forest regressor:

```
X = np.arange(0, 10, 0.5).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

clf = RandomForestRegressor(n_estimators=100, random_state=42)
clf.fit(X, y)
```

- Making prediction:

```
X_test = np.linspace(0, 10, 100).reshape(-1, 1)
y_pred = clf.predict(X_test)
```

- Visualization:

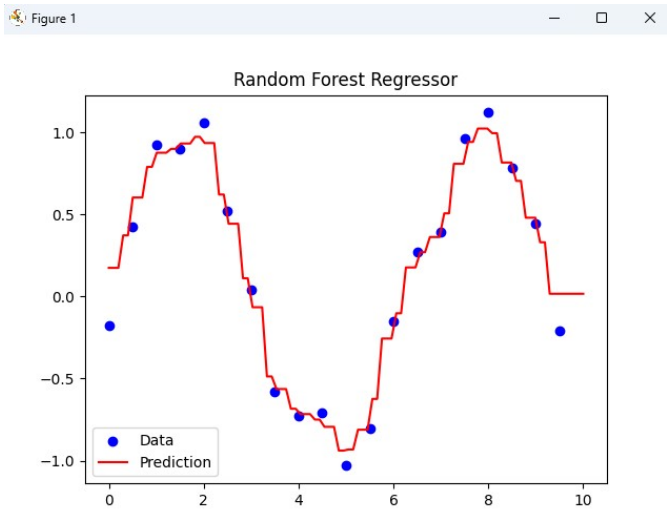

Figure 1.17: Random Forest Regressor model.

2.3.6 K-nearest Neighbors (K-NN):

2.3.6.1 Understanding K-NN:

The k-nearest neighbors algorithm is another non-parametric, supervised learning classifier, which uses proximity to make classifications or predictions about the grouping of an individual data point. It is one of the popular and simplest classification and regression classifiers used in machine learning today [24].

While the K-NN algorithm can be used for either regression or classification problems, it is typically used as a classification algorithm, working off the assumption that similar points can be found near one another [24].

2.3.6.2 How K-NN works:

It stores the training data. When a new point needs to be classified, it looks at th**e** k nearest points (wher*e* k is the number of neighbors the algorithm checks) and assigns the class that appears most often.

The most important step when using K-NN is choosing the right value o*f* k, usually done through *cross-validation* (a technique to evaluate a model's performance by splitting the data into multiple training and testing sets). to find the one that gives the best accuracy. **A** smal*l* k makes the model more sensitive to noise, while **a** large*r* k smooths predictions but can miss finer details [25].

In the next example (Figure 1.18):

- If k = 1, the new data point is classified based on its single closest neighbor.
- If k = 3, it is classified based on the majority class among its 3 nearest neighbors.

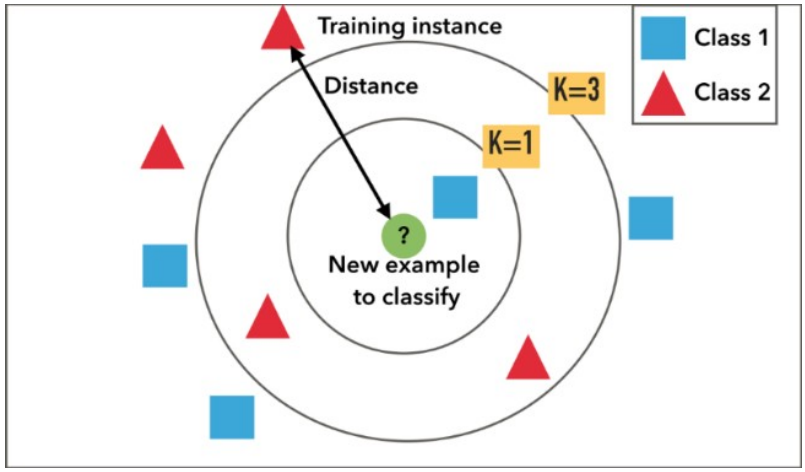

Figure 1.18: How K-NN works [26].

2.3.6.3 Applications of the K-NN algorithm:

- Recommendation Systems.
- Spam Detection.
- Customer Segmentation.
- Speech Recognition [24].

2.3.6.4 Advantages and Disadvantages of K-NN algorithm:

Advantages:

- Simple & easy to implement.
- No training needed it adapts as new data comes in.
- Few hyper-parameters mainly *k* and distance metric.
- works for both classification and regression [24].

Disadvantages:

- Slow with large datasets.
- Struggles with high-dimensional data.
- Can over-fit in complex spaces [24].

2.3.6.5 K-NN in practice:

Example 1 (Classification):

- Importing needed libraries:

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
```

- Loading data and training the model:

  The default value for *k* is 5:

```
X, y = load_iris(return_X_y=True)

clf = KNeighborsClassifier()
clf.fit(X, y)
```

- Making prediction:

```
print(clf.predict([[5.1, 3.5, 1.4, 0.2]]))
```

- Output:

```
[0]
```

The K-NN algorithm predicted that this given new data sample is a *setosa* flower based on the closest 5 neighbors to it.

Example 2 (Regression):

- Importing required libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KneighborsRegressor
```

- Loading the dataset and training the K-Neighbors regressor:

```
X = np.arange(0, 10, 0.5).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

clf = KNeighborsRegressor(n_neighbors=3)
clf.fit(X, y)
```

- Making prediction:

```
X_test = np.linspace(0, 10, 100).reshape(-1, 1)
y_pred = clf.predict(X_test)
```

- Visualization:

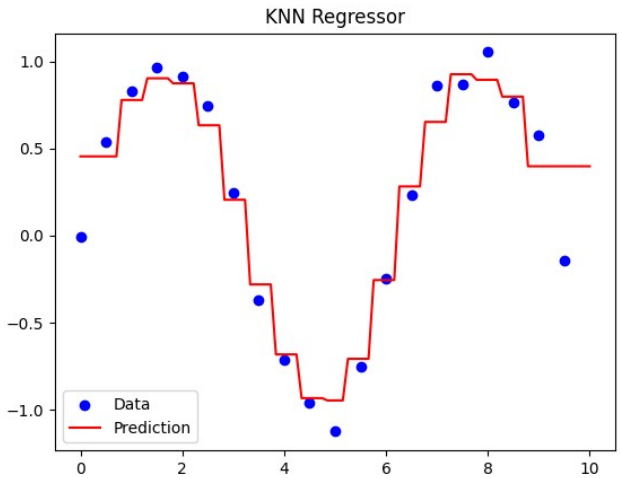

Figure 1.19: KNN Regressor Model.

2.3.7 Neural Networks (NN):

2.3.7.1 Understanding Neural Networks:

A neural network (NN), or artificial neural network (ANN), it is a subset of machine learning, and at the heart of deep learning models [27], typically it has three-layer network with one input layer, one or more hidden layers and one output layer. It can be used for both classification and regression problems [2].

Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

- **Neurons**: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections**: Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases**: These parameters determine the strength and influence of connections.
- **Propagation Functions**: Mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule**: The method that adjusts weights and biases over time to improve accuracy [28].

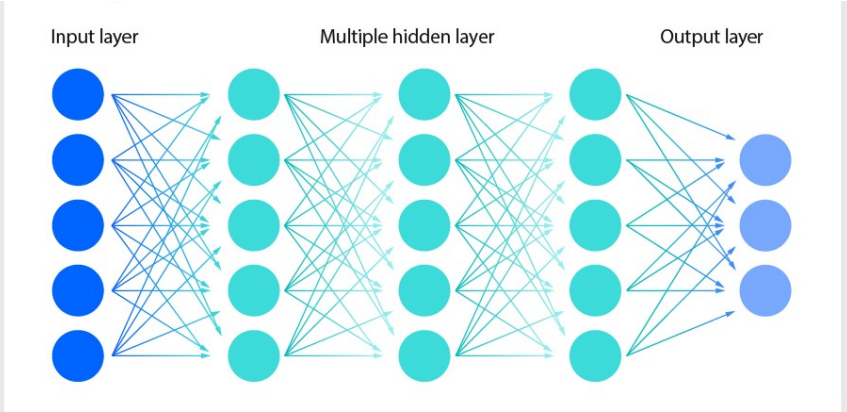

Figure 1.20: Artificial Neural Network [27].

2.3.7.2 Types of Neural Networks:

- **Perceptron**: The earliest neural network, introduced in 1958.
- **Feedforward (MLP)**: Standard networks with input, hidden, and output layers — used in vision, NLP.
- **CNNs**: Specialized for image and pattern recognition using convolution operations.
- **RNNs**: Designed for sequential data with feedback loops — useful for time-series forecasting [27].

2.3.7.3 Applications of Neural Networks:

- **Image/Video Recognition**: Facial recognition, self-driving cars, medical imaging.
- **NLP**: Translation, chat-bots, sentiment analysis.
- **Finance**: Stock prediction, fraud detection.
- **Healthcare**: Disease diagnosis, treatment planning.
- **Autonomous Systems & Gaming**: Real-time decisions in games and self-driving vehicles [28].

2.3.7.4 Advantages and disadvantages of Neural Networks:

Advantages:

- Learns complex patterns from data.
- Handles images, text, and non-linear problems well.
- Supports parallel processing for faster computation.

Disadvantages:

- Needs lots of data and computing power.
- Hard to interpret (“black box”).
- Risk of over-fitting on small datasets [28].

2.3.7.5 Neural Networks in practice:

Example 1 (Classification):

- Importing necessary libraries:

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
```

- Loading the data and training the model:

```
X, y = load_iris(return_X_y=True)

clf = MLPClassifier(max_iter=500)
clf.fit(X, y)
```

- Making prediction:

```
print(clf.predict([[5.1, 3.5, 1.4, 0.2]]))
```

- Output:

```
[0]
```

This Neural Network model again predicts that the newer data sample is a *setosa* flower.

Example 2 (Regression):

- Importing required libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
```

- Loading the dataset and training the MLP Regressor:

```
X = np.arange(0, 10, 0.5).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

clf = MLPRegressor(hidden_layer_sizes=(64, 32), activation='relu',
solver='adam', max_iter=500)
clf.fit(X, y)
```

- Making prediction:

```
X_test = np.linspace(0, 10, 100).reshape(-1, 1)
y_pred = clf.predict(X_test)
```

- Visualization:

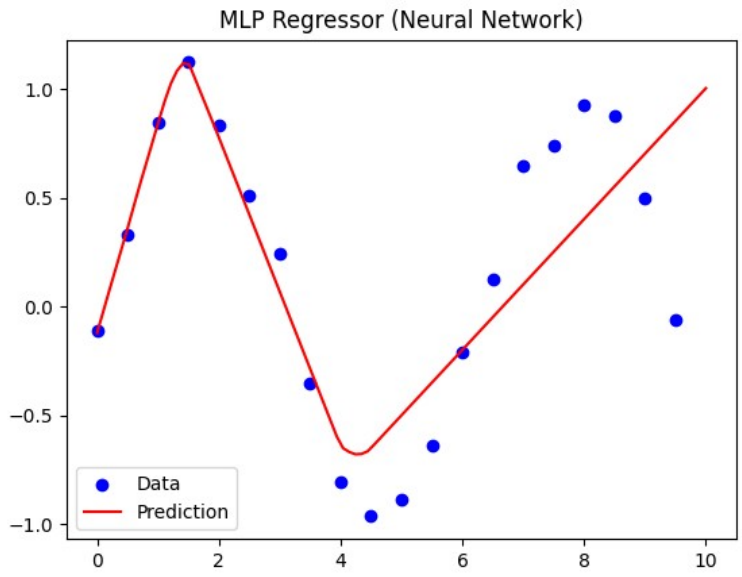

Figure 1.21: Neural Network Regressor Model.

CHAPTER III: Unsupervised Learning

3. What is Unsupervised Learning?

Unsupervised Learning is a type of machine learning technique that does not require you to provide knowledge to supervise the model. The model will discover information on its own shown in Figure 3.1, it is useful when you have a large amount of unlabeled data; it can be used for applications such as clustering, association, dimensionality reduction, etc [2].

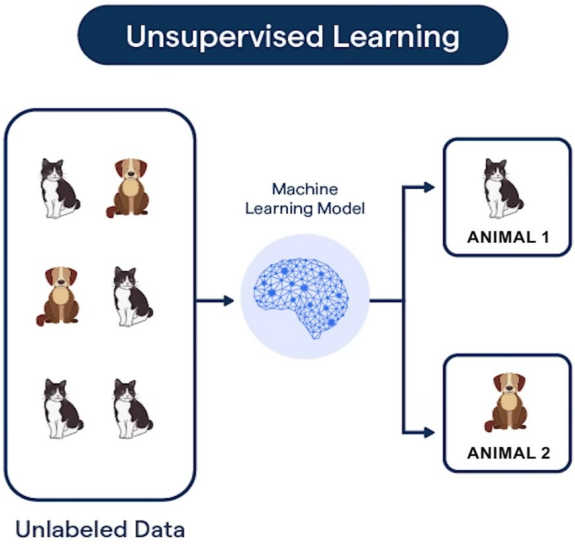

Figure 3.1: Unsupervised Learning [6].

3.1 Clustering:

3.1.1 Definition of Clustering:

Clustering is an unsupervised machine learning technique that groups similar data points based on shared features or patterns as shown in Figure 3.2. It's widely used for exploratory data analysis, pre-processing, anomaly detection, and reducing data complexity [29].

Clustering reveals hidden structures in data, such as customer segments or social groups, helping to better understand or simplify large datasets. It includes hard clustering, where each point belongs to one cluster, and soft clustering, where points have probabilistic membership [29].

Common uses include finding trends, detecting outliers, and enhancing data visualization. The best algorithm depends on your data and goals [29].

Some well-known clustering methods are Centroid-based, Hierarchical, Distribution-based, Density-based, and Grid-based clustering [29].

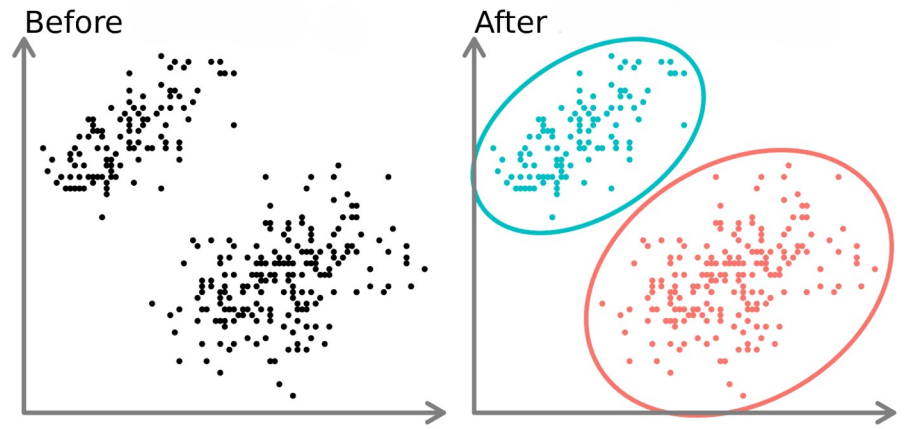

Figure 3.2: Before and After Clustering [30].

3.1.2 K-Means Clustering Algorithm:

3.1.2.1 Understanding K-Means Clustering:

K-means clustering is one the most commonly used clustering algorithms. It is an iterative algorithm that helps find a number of clusters for a given dataset.

3.1.2.2 How K-Means work:

K-means clustering groups data points by minimizing the distance between points and their cluster centers, called centroids. It starts by choosing k centroids, either randomly or by sampling.

Then, it repeats two steps: assign each point to the nearest centroid (expectation), and update each centroid to the mean of its assigned points (maximization). This continues until the centroids stabilize or a limit is reached in the next figure we set $k=2$ shown in Figure 3.3.

K-means is simple but sensitive to how centroids are initialized and to outliers. Choosing the right number of clusters and good initialization is important for meaningful results [31].

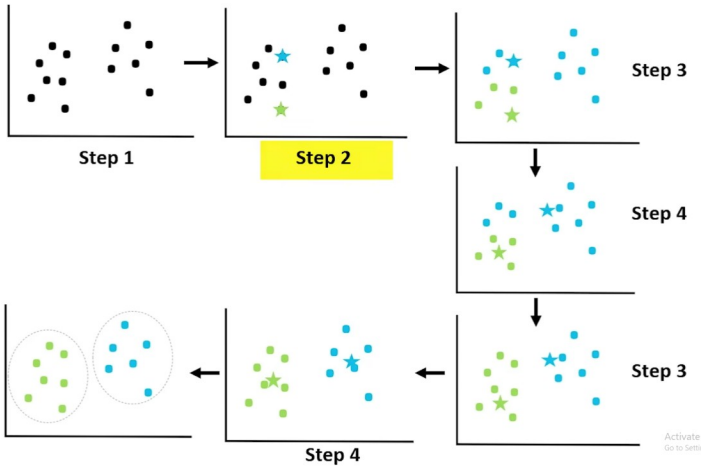

Figure 3.3: K-means Clustering Steps [6].

3.1.2.3 Applications of K-Means:

- Anomaly detection.
- Market research.
- Image segmentation.
- Document processing.

3.1.2.4 Advantages and disadvantages of K-Means:

Advantages:

- Simple to understand and implement, most popular unsupervised method.
- Fast and computationally efficient compared to hierarchical clustering.
- Scales well to large datasets and handles various cluster shapes and sizes.

Disadvantages:

- Depends heavily on number of clusters and initial centroids, poor choices affect results.
- Performs poorly on uneven cluster sizes, outliers, complex shapes, and high-dimensional data.

3.1.2.5 K-Means in practice:

- Importing required libraries:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

- Preparing data and training the model:

```
X = [
    [1, 2], [1, 4], [2, 1], [2, 3], [0, 2], [1, 1], # Cluster 0 (top-left)
    [8, 7], [9, 8], [7, 9], [8, 8], [9, 6], [10, 7],# Cluster 1 (bottom-right)
    [3, 8], [4, 9], [2, 7], [3, 7], [4, 8], [5, 7]  # Cluster 2 (top-center)
]

                #k=3 clusters
kmeans = KMeans(n_clusters=3, random_state=0, n_init='auto').fit(X)
```

- Visualization:

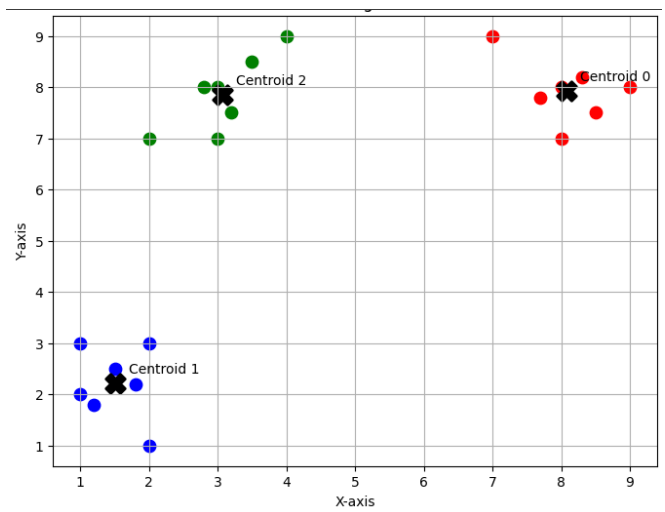

Figure 3.4: K-means plot.

3.2 Association:

3.2.1 Definition of Association:

Association, also known as association rule learning or mining is a common technique used to discover associations in unsupervised machine learning. This technique is a rule-based ML technique that finds out some very useful relations between parameters of a large data set. This technique is basically used for market basket analysis that helps to better understand the relationship between different products [32].

For example shopping stores use algorithms based on this technique to find out the relationship between the sale of one product with respect to another's sales based on customer behavior, for example if a customer buys milk, then he may also buy bread, eggs, or butter (Figure 3.5). Once trained well, such models can be used to increase their sales by planning different offers [32].

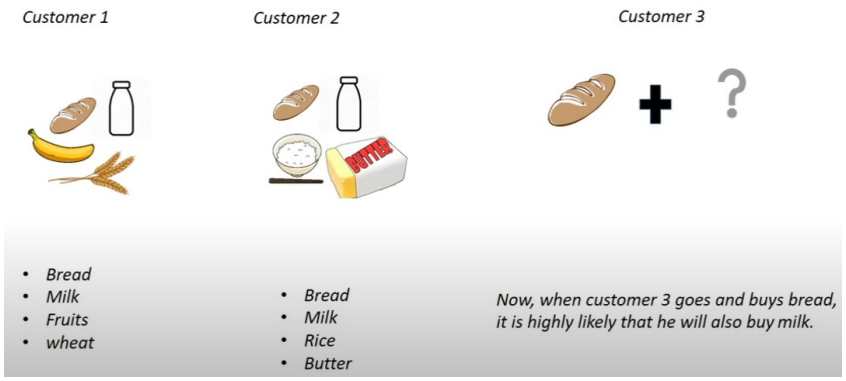

Figure 3.5: Using Association to analyze customer's behavior [33].

3.2.2 Apriori Algorithms:

3.2.2.1 Understanding Apriori algorithm:

This algorithm counts the frequency of co-occurring items, or associations, in large datasets with potentially thousands of attributes. The goal is to discover patterns that occur significantly more often than would be expected by random chance [34].

3.2.2.2 How Apriori algorithm works:

To quantify these associations, several key metrics are used:

- **Support**: Measures how frequently an itemset appears in the dataset. It reflects the popularity of the item-set.

$$Support(A) = \frac{Number\ of\ transactions\ containing\ A}{Total\ number\ of\ transactions}$$

- **Confidence**: Measures how often item B is purchased when item A is purchased:

$$Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}$$

- **Lift**: Measures how likely item B is purchased when item A is purchased, relative to how often B is purchased independently.

$$Lift(A \rightarrow B) = \frac{Confidence(A \rightarrow B)}{Support(B)}$$

Lift can be interpreted as follows:

- **Lift = 1** → A and B occur independently.
- **Lift > 1** → A and B are positively correlated.

- **Lift < 1** → A and B are negatively correlated; A may act as a substitute for B [34].

3.2.2.3 Applications of Apriori algorithm:

- **E-commerce:** Recommends frequently co-purchased products.
- **Food Delivery Services:** Identifies popular food combos.
- **Streaming Services:** Suggests related movies/shows.
- **Financial Services:** Analyzes spending habits to offer personalized deals [35].

3.2.2.4 Advantages and disadvantages of Apriori algorithm:

Advantages:

- Easy to implement and understand
- Works well on large transactional datasets
- Helps uncover hidden patterns and relationships
- Useful for decision-making in marketing and sales

Disadvantages:

- Generates a large number of rules, many of which may be irrelevant
- Requires careful tuning of support and confidence thresholds
- Doesn't handle continuous data without preprocessing
- Assumes independence between item occurrences

3.2.2.5 Apriori algorithm in practice:

The following Python code demonstrates how to use the Apriori algorithm to extract frequent itemsets and generate association rules using a small basket dataset. The `mlxtend` library handles preprocessing, mining, and rule generation.

- Importing necessary libraries for data manipulation, visualization, and association mining:

```
import pandas as pd
import matplotlib.pyplot as plt
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

- Defining a list of transactions (items bought together):

```
transactions = [
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Bread', 'Eggs'],
    ['Bread', 'Eggs', 'Chips'],
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Bread', 'Chips']
]
```

- Converting the list of transactions to boolean table:

```
te = TransactionEncoder()
te_array = te.fit_transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)
```

- Finding frequent items (appearing in at least 40% of baskets):

```
frequent_items = apriori(df, min_support=0.4, use_colnames=True)
```

- Generating meaningful rules:

```
rules = association_rules(frequent_items, metric="confidence",
min_threshold=0.6)
```

- Output:

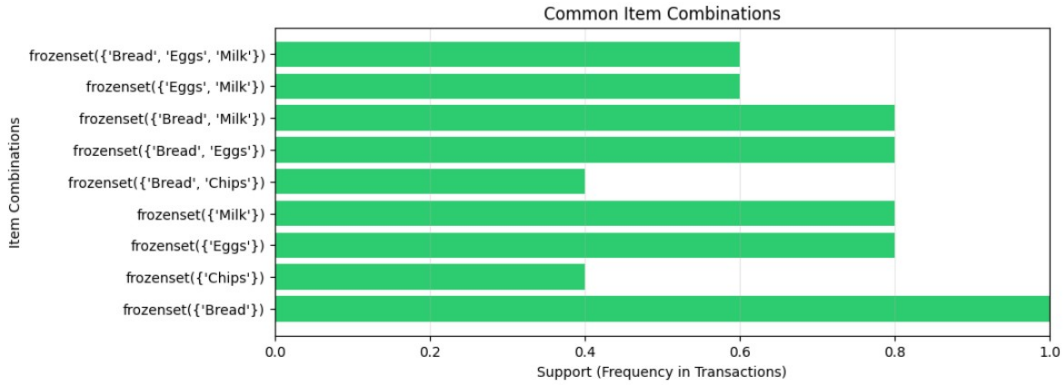

Figure 3.6: Support value for item combinations.

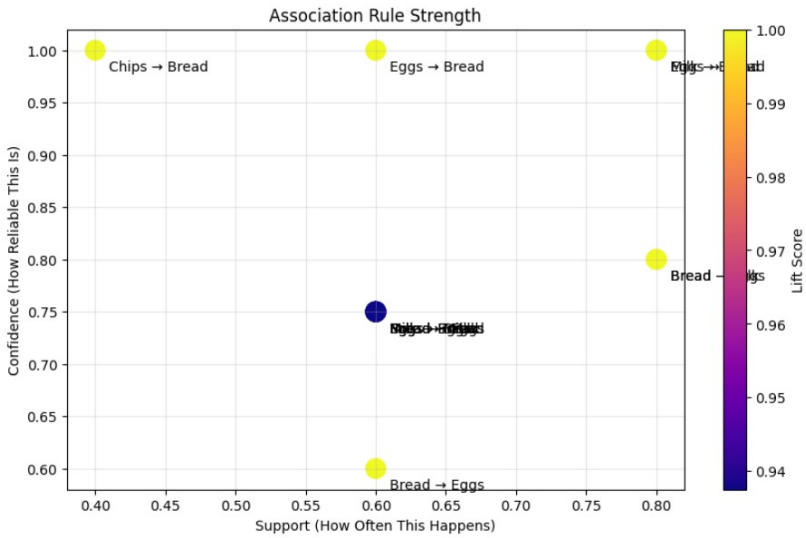

Figure 3.7: Confidence and Lift score for item combinations.

Rule	Frequency (Support)	Reliability (Confidence)	Significance (Lift)
Milk → Bread	4/5 baskets	100%	1.0x (baseline)
Eggs → Bread	4/5 baskets	100%	1.0x
Chips → Bread	2/5 baskets	100%	1.0x
Milk+Eggs → Bread	3/5 baskets	100%	1.0x

Table 3.1: Support, Confidence and significance for each item combination.

3.3 Dimensionality Reduction:

3.3.1 Definition of Dimentionality Reduction:

Real-world data like speech, images, or fMRI scans often have high dimensionality, which can complicate analysis. Dimensionality reduction transforms such data into a lower-dimensional, meaningful form that reflects its intrinsic structure—the minimal number of variables needed to describe it. This process helps overcome issues like the curse of dimensionality and supports tasks such as classification, visualization, and compression. Traditional linear methods include PCA, factor analysis, and classical scaling, though they struggle with complex nonlinear data [36].

3.3.2 Principle Component Analysis (PCA):

3.3.2.1 Understanding PCA:

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms correlated variables into a smaller set of uncorrelated variables called principal components. Originally from multivariate data analysis, PCA is widely used in areas like signal de-noising, blind source separation, and data compression [14].

By projecting data into a lower-dimensional space, PCA simplifies the interpretation of complex datasets, making it easier to identify patterns, trends, and outliers. It's often the first step in analyzing large datasets and is considered one of the key results of applied linear algebra [14].

PCA itself does not perform predictions. For classification or prediction, algorithms like Linear Discriminant Analysis (LDA) are needed. PCA can preprocess the data, but LDA is responsible for making the actual predictions.

3.3.2.2 How PCA works:

The goal is also to project a dataset onto a lower-dimensional space. LDA aims to create new axes (called discriminant) to maximize class separation, while PCA aims to find new axes (called components) to maximize variance (the average of the squared difference form the mean) [2].

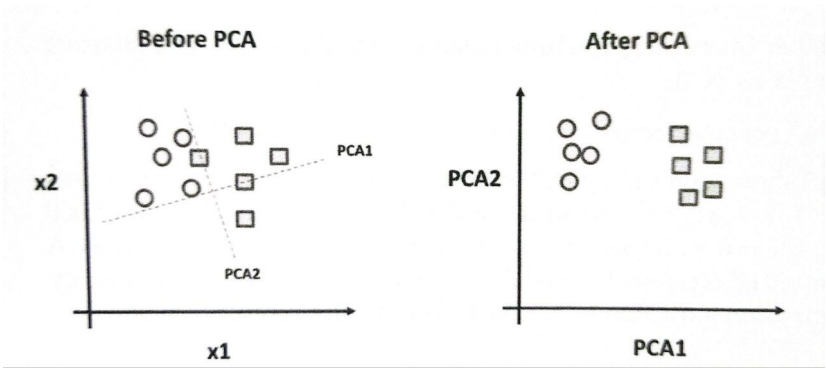

Figure 1.11: Data before and after applying PCA [2].

3.3.2.3 Applications of PCA:

- Image compression.
- Data visualization.
- Noise filtering [15].

3.3.2.3 Advantages and disadvantages of PCA:

Advantages:

- Reduces data size and speeds up analysis.
- Minimizes over-fitting and noise.
- Helps visualize complex data [15].

Disadvantages:

- Some information is lost.
- New features are hard to interpret.
- Assumes linear relationships and needs scaled data.
- Sensitive to outliers [15].

3.3.2.4 PCA in practice:

- Importing required libraries:

```
from sklearn import datasets
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

- Loading the Iris dataset:

```
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

- initializing and training the PCA model:

```
# the next line reduces the dataset's dimensionality to 3 components.
pca = PCA(n_components=3)
pca.fit(X)
X_pca = pca.transform(X)
```

- Initializing and training the LDA model using the PCA new transformed data:

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_pca, y)
```

- Making prediction:

```
prediction = lda.predict([pca.transform([[5.0, 3.4, 1.5, 0.4]])])
print(prediction[0])
```

- Output:

```
[0]
```

After applying *dimensionality reduction* using PCA we the predicted class for the given data sample is still setosa.

- Visualization:

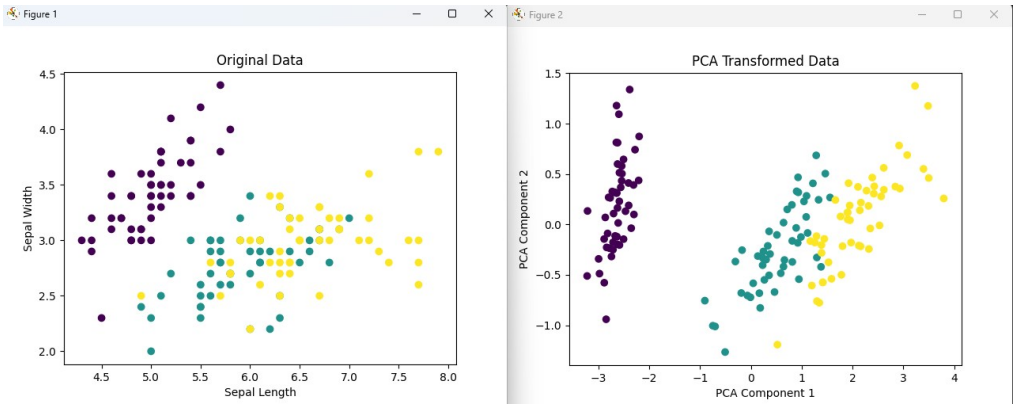

Figure 1.12: The Iris dataset before and after applying PCA.

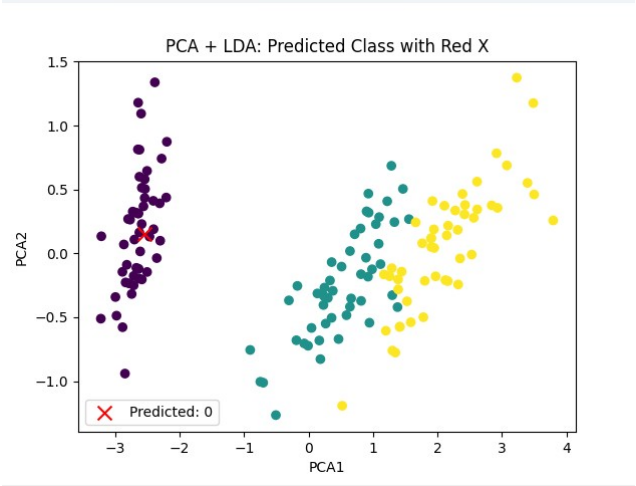

Figure 1.13: The predicted class for the given data sample PCA+LDA.

Conclusion:

This thesis presented an overview of key supervised and unsupervised machine learning algorithms, focusing on their theoretical foundations and practical implementations. Through simplified examples and Python-based demonstrations, it explored how various algorithms operate and how they can be applied effectively to real-world problems.

Supervised learning remains the dominant paradigm due to its reliance on labeled data and clear evaluation metrics. Meanwhile, unsupervised learning techniques—despite their challenges—play a vital role in data exploration, pattern recognition, and preprocessing.

Looking ahead, this work opens opportunities for building smarter, more efficient systems. Future improvements could include adding multiple decision outputs for multiclass predictions, integrating wireless communication (e.g., Wi-Fi) to replace serial interfaces, and employing multi-angle camera setups or infrared sensors to improve detection reliability. Visual or audio feedback, such as status LEDs or buzzers, could also enhance usability in practical deployments.

As machine learning continues to evolve, mastering these foundational techniques provides the tools necessary to explore more complex models, larger datasets, and real-time applications in both academic and industrial settings.

References

[1] A. V. Joshi, *Machine Learning and Artificial Intelligence*, 2nd ed. Cham: Springer, 2023. doi: 10.1007/978-3-031-12282-8.

[2] P. Xiao, *Artificial Intelligence Programming with Python: From Zero to Hero*, 1st ed. Hoboken, NJ: Wiley, 2022.

[3] G. Gautam, "Machine Learning 101," accessed May 5, 2025. [Online]. Available: https://gautam-j.github.io/machine-learning-101

[4] A. S. Nyamawe, M. M. Mjahidi, N. E. Nnko, S. A. Diwani, G. G. Minja, and K. Malyango, *Practical Machine Learning: A Beginner's Guide with Ethical Insights*. Chapman and Hall/CRC Press, 2025. doi: 10.1201/9781003486817.

[5] A. S. Nyamawe, M. M. Mjahidi, N. E. Nnko, S. A. Diwani, G. G. Minja, and K. Malyango, *Practical Machine Learning: A Beginner's Guide with Ethical Insights*, 1st ed. Boca Raton, FL: CRC Press, 2025, doi: 10.1201/9781003486817.

[6] Infinite Codes, "All machine learning algorithms explained in 17 min," *YouTube*, Aug. 17, 2020. [Video]. Available: https://www.youtube.com/watch?v=E0Hmnixke2g

[7] GeeksforGeeks, "Least Square Method," accessed May 5, 2025. [Online]. Available: https://media.geeksforgeeks.org/wp-content/uploads/20240611190240/Least-Square-Method-01.webp

[8] EDUCBA, "Non-Linear Regression," accessed May 5, 2025. [Online]. Available: https://cdn.educba.com/academy/wp-content/uploads/2023/02/Non-Linear-Regression.jpg

[9] Scikit-learn developers, "Support Vector Machines," *Scikit-learn documentation*, accessed May 6, 2025. [Online]. Available: https://scikit-learn.org/stable/modules/svm.html

[10] IBM, "Support Vector Machine (SVM)," *IBM Think*, accessed May 6, 2025. [Online]. Available: https://www.ibm.com/think/topics/support-vector-machine

[11] IBM, "Naive Bayes," *IBM Think*, accessed May 6, 2025. [Online]. Available: https://www.ibm.com/think/topics/naive-bayes

[12] S. Balakrishnama and A. Ganapathiraju, "Linear Discriminant Analysis – A Brief Tutorial," *Institute for Signal and Information Processing*, [Online]. Available: http://www.isip.msstate.edu/

[13] IBM, “Linear Discriminant Analysis,” *IBM Think*, accessed May 6, 2025. [Online]. Available: https://www.ibm.com/think/topics/linear-discriminant-analysis

[14] M. Richardson, “Principal Component Analysis,” May 2009.

[15] IBM, “Principal Component Analysis,” *IBM Think*, accessed May 8, 2025. [Online]. Available: https://www.ibm.com/think/topics/principal-component-analysis

[16] EDUCBA, “Decision Tree Algorithm,” accessed May 8, 2025. [Online]. Available: https://www.educba.com/decision-tree-algorithm/

[17] B. T. Jijo and A. M. Abdulazeez, “Classification based on decision tree algorithm for machine learning,” *Duhok Polytechnic University*, 2021. doi: 10.25007/ajnu.v10n1a1035.

[18] GeeksforGeeks, “Decision Tree,” accessed May 8, 2025. [Online]. Available: https://www.geeksforgeeks.org/decision-tree/

[19] IBM, “Decision Trees,” *IBM Think*, accessed May 8, 2025. [Online]. Available: https://www.ibm.com/think/topics/decision-trees

[20] IBM, “Random Forest,” *IBM Think*, accessed May 8, 2025. [Online]. Available: https://www.ibm.com/think/topics/random-forest

[21] R. Yeho, “Random Forests,” *Medium*, accessed May 8, 2025. [Online]. Available: https://medium.com/@roiyeho/random-forests-98892261dc49

[22] GeeksforGeeks, “Random Forest Algorithm in Machine Learning,” accessed May 8, 2025. [Online]. Available: https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/

[23] GeeksforGeeks, “What are the Advantages and Disadvantages of Random Forest?,” accessed May 9, 2025. [Online]. Available: https://www.geeksforgeeks.org/what-are-the-advantages-and-disadvantages-of-random-forest/

[24] IBM, “K-Nearest Neighbors (KNN),” *IBM Think*, accessed May 12, 2025. [Online]. Available: https://www.ibm.com/think/topics/knn

[25] GeeksforGeeks, “K-Nearest Neighbours,” accessed May 12, 2025. [Online]. Available: https://www.geeksforgeeks.org/k-nearest-neighbours/

[26] DeepAI, “K-Nearest Neighbors (kNN),” accessed May 12, 2025. [Online]. Available: https://deepai.org/machine-learning-glossary-and-terms/kNN

[27] IBM, “Neural Networks,” *IBM Think*, accessed May 13, 2025. [Online]. Available: https://www.ibm.com/think/topics/neural-networks

[28] GeeksforGeeks, “Neural Networks: A Beginner’s Guide,” accessed May 13, 2025. [Online]. Available: https://www.geeksforgeeks.org/neural-networks-a-beginners-guide/

[29] IBM, “Clustering,” *IBM Think*, accessed May 14, 2025. [Online]. Available: http://ibm.com/think/topics/clustering

[31] GeeksforGeeks, “K-Means Clustering Introduction,” accessed May 14, 2025. [Online]. Available: https://www.geeksforgeeks.org/k-means-clustering-introduction/

[32] GeeksforGeeks, “Unsupervised Learning,” accessed May 14, 2025. [Online]. Available: https://www.geeksforgeeks.org/unsupervised-learning/

[33] Siddhardhan, “Unsupervised Learning | Clustering and Association Algorithms in Machine Learning,” *YouTube*, accessed May 14, 2025. [Video]. Available: https://www.youtube.com/watch?v=d9qf0qPPhsc&t=357s

[34] F. Erlandsson, P. Bródka, A. Borg, and H. Johnson, “Finding influential users in social media using association rule learning,” *Entropy*, vol. 18, no. 5, p. 164, 2016. doi: 10.3390/e18050164.

[35] GeeksforGeeks, “Apriori Algorithm,” accessed May 16, 2025. [Online]. Available: https://www.geeksforgeeks.org/apriori-algorithm/

[36] L. van der Maaten, E. Postma, and J. van den Herik, *Dimensionality Reduction: A Comparative Review*, TiCC TR 2009–005, Tilburg Centre for Creative Computing, Tilburg University,

Oct. 2009. [Online]. Available: http://www.uvt.nl/ticc